

Week 4 Portfolio Milestone in C

John Wensink

CSC507 Foundations of Operating Systems

Colorado State University – Global Campus

Dr. Joseph Issa

December, 8th, 2024

Week 4 Portfolio Milestone in C

This week, we explored four different strategies for processing large datasets, with the task of doubling each line of numbers in large and very large files. The goal was to evaluate the performance, efficiency, and scalability of the different methods.

Method 0: Bash Script

For the first method, we created a simple bash script to multiply each line of a one-million-line file, by two. To confirm that `file1.txt` was indeed one million lines long, we ran a simple terminal command `wc -l file1.txt`, which confirmed that the file had the correct amount of lines.

```
jwensink@vbox:~$ wc -l file1.txt
1000000 file1.txt
jwensink@vbox:~$
```

The general idea for the script was provided on the assignment page (CSUGlobal, 2024) and I used an LLM Grimoire (2024) to format the spacing and syntax. The script processes each line of the file sequentially, using a *while* loop to read each number, double it, and append the result to the output file *newfile1.txt*. While this is an effective approach for small datasets, the bash script was by far the slowest and least efficient approach, taking approximately 124 seconds to finish the process. This is likely due to the overhead of reading and writing each line individually in an interpreted language. This inefficiency was made worse by the lack of low-level optimizations available in compiled languages like C. The sequential execution and the frequent I/O operations

resulted in significant performance bottlenecks. For large datasets, this method is impractical due to its prolonged runtime.

Method 1: Full-Memory Processing in C

The first C-based approach was designed to read the entire contents of a file into memory, process all the data in a single operation, and write the results to a new output file. This method will take advantage of the speed of batch processing and low-level efficiencies in a compiled language but still requires substantial system memory to accommodate the entire dataset. I used the LLM Grimoire (2024) to help me generate the program. This VM implementation is running on 16GB of DDR4 RAM, and 4 physical CPU cores, each with its own L1 and L2, cache, but the L3 cache is shared across all cores. Still, 1 million rows of integers require about 4MB of memory (assuming each integer is 4 bytes), which may very well exceed the size of the CPU caches, which are typically measured in KB or MB. As a result, I expected most of the processing to occur in system RAM. But sure enough, the lines were multiplied in 0.14 seconds.

```
jwensink@vbox:~$ gcc -o week4_step1 week4_step1.c
jwensink@vbox:~$ ./week4_step1
Processing completed in 0.14 seconds.
jwensink@vbox:~$ wc -l newfile1.txt
1000000 newfile1.txt
jwensink@vbox:~$ head file1.txt newfile1.txt
==> file1.txt <==
14541
9621
23914
32149
14247
21943
9664
30130
12553
28916

==> newfile1.txt <==
29082
19242
47828
64298
28494
43886
19328
60260
25106
57832
jwensink@vbox:~$
```

This suggests to me that perhaps the system stored at least some of file1.txt in cache. Spoilers ahead: Methods 2 and 3 also performed at similar speeds with 1M rows of numbers to multiply. It seemed like it may be difficult to compare the methods without either a restriction on resources or an increase in the file size. Like last week, I decided to increase the file size to 1B lines to get a better comparison of how the processes were performing. I modified the C program in Method 1 to create a new file named *newfile1billion.txt* and verified the line count was correct. Method 1 took 216.35 seconds to complete this calculation, suggesting that perhaps the system might have needed to make use of virtual memory to load the full ~4GB file into memory to perform this operation.

```
jwensink@vbox:~$ gcc -o week4_step1billion week4_step1billion.c
jwensink@vbox:~$ ./week4_step1billion
Processing completed in 216.35 seconds.
jwensink@vbox:~$
```

Method 2: Reading a Single Row at a Time

The second approach processed the file line by line. Instead of reading the entire dataset into memory, this method read each number sequentially, doubled it, and immediately wrote the result to the output file. This approach would significantly reduce memory usage, with only a single number being stored in memory at a given time. Grimoire (2024) helped again with the implementation of the C program. The program uses *fscanf()* to read a single integer from the file and avoids loading the entire file into memory. After reading the number (*number *= 2*) multiply it, and *fprintf()* prints the file to text. Temporary file I/O buffers (which are small) are managed by the system this time and not by the program itself as I did in Week 3's milestone.

Again, the entire computation took only 0.13 seconds, which is only 0.01 seconds faster than Method 1's approach.

```
jwensink@vbox:~$ nano week4_step2.c
jwensink@vbox:~$ gcc -o week4_step2 week4_step2.c
jwensink@vbox:~$ ./week4_step2
Processing completed in 0.13 seconds.
jwensink@vbox:~$ wc -l newfile1.txt
1000000 newfile1.txt
jwensink@vbox:~$ head file1.txt newfile1.txt
==> file1.txt <==
14541
9621
23914
32149
14247
21943
9664
30130
12553
28916

==> newfile1.txt <==
29082
19242
47828
64298
28494
43886
19328
60260
25106
57832
jwensink@vbox:~$
```

So, to test the scalability of method 2, I ran it again on newfile1billion.txt. This took 199.90 seconds to execute, which was a bit faster (16.46, or about 7.61%) than method 1's 216.35 seconds. Not too bad

```
jwensink@vbox:~$ ./week4_step2billion
Processing completed in 199.90 seconds.
jwensink@vbox:~$
```

Method 3: File Splitting

The third approach, implemented in C, involves striking a balance between the memory efficiency of Method 2's line-by-line approach and the batch processing speed of Method 1 by splitting the input file into more manageable chunks. Instead of processing the file line by line or loading the entire dataset into memory, this method split the dataset in half to work on 500M

lines at a time. This probably doesn't make much difference compared to method 1 as it's still a large dataset, but it does reduce the memory requirement by half, which could make an impact if the system had limited RAM memory. Since both chunks are processed sequentially, I would expect the time savings to arise from improved memory management and reduced strain on virtual memory. Like method 1, the program benefits from fewer I/O operations (running at SATA speeds) compared to option 2, as each chunk is loaded and processed in 1 go before moving to the next. Grimoire (2024) again helped me with the implementation, splitting the dataset in two using ***long midpoint = total_lines / 2;***. The program processes each half of the file by reading one line at a time, doubling the integer, and writing the output to the file similarly to method 2. The program processes each half of the file, 2 lines at a time. By re-using the file pointer, ***fseek(input_file, 0, SEEK_SET);*** ensures that both halves are processed independently without needing to split the file into physical parts. As expected, method 3 took 0.14 seconds to process 1M lines per the assignment. For 1B lines, splitting the data into 2 chunks took 168.18 seconds, which is 22.26% faster than method 1, and 15.87% faster than method 2.

```
jwensink@vbox:~$ ./week4_step3billion
Processing completed in 168.18 seconds.
jwensink@vbox:~$
```

I was curious about what would happen if I split the data up into 8 chunks and ran that simultaneously for each of the 8 logical processors in the 4 physical cores. It turns out that this took 164.20 seconds, a bit of an improvement, but not dramatic compared to dividing it into two chunks. About 2.37% better, which surprised me, I thought it would be a much more substantial improvement.

```
jwensink@vbox:~$ ./week4_step3billion
Processing completed in 164.20 seconds.
jwensink@vbox:~$
```

Conclusion

This week's milestone provided interesting insights into the performance, efficiency, and scalability of the four different approaches for processing large and very large datasets using bash and C. The bash script was simple but was by far the least efficient, with significant runtime and overhead due to line-by-line interpretation and frequent I/O operations. It's clear that bash is not well suited to handle datasets beyond a few thousand rows. In fact, I did not even attempt to feed it 1B lines. The C programs showed how incredibly fast a compiled language can be, achieving a runtime of just a fraction of a second for all three methods using 1M lines. Method 1, requiring the full dataset to be loaded into memory, pushed my system to the limit with 1B lines of code, and it was interesting to see that it took the longest of the three compiled methods to complete. Method 2's line-by-line approach was a significant improvement, making it a scalable solution for very large files like *file1billion.txt*. However, frequent I/O operations introduced a bottleneck, making it better than method 1 but not the best it could be. Option 3 really shined. By striking a balance by splitting the dataset into halves (and then eights), this approach improved the memory efficiency, leading to significant performance improvements for very large datasets. I thought that by dividing the dataset into 8 chunks, the performance would be markedly improved over 2 chunks, but it was only slightly better. I don't know why this is, but I suspect that disk I/O speeds are the limiting factor of method 3. In summary, these experiments emphasized the importance of designing algorithms that match the dataset and system architecture, especially for very large files. I deepened my understanding of C's efficiency and

learned about the important role of resource-aware programming in operating systems and software development.

References

CSUGlobal. (2024). Week 4 Portfolio Milestone. Retrieved December 8, 2024, from:
https://csuglobal.instructure.com/courses/102374/assignments/1925435?module_item_id=5271639

Eyerman, S., Du Bois, K., & Eeckhout, L. (2012). Speedup stacks: Identifying scaling bottlenecks in multi-threaded applications. Retrieved December 1, 2024, from:
https://users.elis.ugent.be/~leeckhou/papers/ispass12_2.pdf

GeeksforGeeks. (n.d.). Multithreading in C. Retrieved November 30 and December 8, 2024, from: <https://www.geeksforgeeks.org/multithreading-in-c/>

Grimoire. (2024). Conversation about the C programming language. OpenAI. Retrieved November 30, and December 8, 2024, from: <https://chat.openai.com>

Stallings, W. (2018). Operating systems: Internals and design principles (9th ed., Global ed.). Pearson. Retrieved November 30 and December 8, 2024, from
https://www.youseficlass.ir/wp-content/uploads/2023/07/William_Stallings_Operating_Systems_Internals_and_Design_Principles_watermark.pdf

TutorialsPoint. (n.d.-a). C Library - <time.h>. Retrieved November 30 and December 8, 2024, from https://www.tutorialspoint.com/c_standard_library/time_h.htm

TutorialsPoint. (n.d.-b). C Library - <limits.h>. Retrieved November 30 and December 8, 2024, from https://www.tutorialspoint.com/c_standard_library/limits_h.htm